Manipal University

Name: Devanshi Rana
Email: rana.2502051097@muj.manipal.edu
Roll no: 2502051097
Phone: 7600927404
Branch: Manipal
Department: CSE - T1 - Batch 2
Batch: 2029
Degree: B.Tech

Scan to verify results

NeoColab_Manipal_2029_DSA Lab Course

Neocolab_Manipal_DSA_Week 8_PAH

Attempt : 1
Total Mark : 20
Marks Obtained : 20

Section 1 : coding

1. Problem Statement

Given a string s representing an infix expression ("operand1 operator operand2"), Convert it into its postfix notation ("operand1 operand2 operator").

Note: The precedence order is as follows: (^) has the highest precedence and is evaluated from right to left, (* and /) come next with left to right associativity, and (+ and -) have the lowest precedence with left to right associativity.

Examples:

Input: s = "a*(b+c)/d"

Output: abc+*d/

Explanation: The expression is a * (b + c) / d. First, inside the brackets, b + c becomes bc+. Now the expression looks like a * (bc+) / d. Next, multiply a with (bc+), so it becomes abc+* . Finally, divide this result by d, so it becomes abc+*d/.

Input: s = "a+b*c+d"

Output: abc*+d+

Explanation: The expression a + b * c + d is converted by first doing b * c bc*, then adding a abc*+, and finally adding d abc*+d+.

Input Format

The input consists of a single line containing a valid infix expression as a string

The expression may include:

- Operands: lowercase/uppercase alphabets or digits.
- Operators: +, -, *, /, ^.
- Parentheses: (and).

Output Format

The output prints the equivalent postfix expression on a single line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: a*(b+c)/d

Output: abc+*d/

Answer

```
// You are using GCC
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX 100
```

```
char stack[MAX];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
char peek() { return stack[top]; }
int isEmpty() { return top == -1; }

int precedence(char op) {
   if(op == '^') return 3;
   if(op == '*' || op == '/') return 2;
   if(op == '+' || op == '-') return 1;
   return 0;
}

int isRightAssociative(char op) {
   return op == '^';
}

void infixToPostfix(char* exp) {
   char output[MAX] = "";
   int k = 0;
   for(int i = 0; exp[i]; i++) {
      char c = exp[i];
      if(isalnum(c)) {
         output[k++] = c;
      } else if(c == '(') {
         push(c);
      } else if(c == ')') {
         while(!isEmpty() && peek() != '(')
            output[k++] = pop();
         pop(); // remove '('
      } else { // operator
         while(!isEmpty() && precedence(peek()) >= precedence(c) &&
              !(isRightAssociative(c) && precedence(peek()) == precedence(c))) {
            output[k++] = pop();
         }
         push(c);
      }
   }
   while(!isEmpty()) output[k++] = pop();
   output[k] = '\0';
```

```
    printf("%s\n", output);
}

int main() {
    char exp[MAX];
    scanf("%s", exp);
    infixToPostfix(exp);
    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem statement

Kiran collects tokens at a museum. However, visitors do not arrive in the order of token numbers, causing the tokens to be jumbled. At the end of his shift, he must write the next greater token number on the back of each token. If no greater number exists, he writes -1.

Manually finding the next greater number is time-consuming, so Kiran, a part-time programmer, decides to implement this process using stacks.

Example

Input

5

2 7 10 19 1

Output

2 7

7 10

10 19

19 -1

1 -1

Explanation

For 2, the next greater is 7For 7, the next greater is 10For 10, the next greater is 19For 19, there is no next greater element so -1For 1, there is no next greater element so -1.

Input Format

The first line contains an integer N — the number of tokens.

The second line contains N space-separated integers representing the token numbers.

Output Format

The output prints the token number for each token followed by its next greater token number. If no greater number exists, print -1.

Each output pair should be printed on a new line.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 6
57 3 4 99 19 50
Output: 57 99
3 4
4 99
99 -1
19 50
50 -1

Answer

```
// You are using GCC
#include <stdio.h>

#define MAX 100

int main() {
```

```
    int n;
    scanf("%d", &n);
    int arr[MAX], nge[MAX], stack[MAX];
    int top = -1;

    for(int i = 0; i < n; i++) scanf("%d", &arr[i]);

    // Traverse from right to left
    for(int i = n-1; i >= 0; i--) {
        while(top != -1 && stack[top] <= arr[i]) top--; // pop smaller/equal
        if(top == -1) nge[i] = -1;
        else nge[i] = stack[top];
        stack[++top] = arr[i]; // push current
    }

    for(int i = 0; i < n; i++) {
        printf("%d %d\n", arr[i], nge[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***